

AIE683 Machine Learning

Week 8: Model Selection & Experiment Tracking

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

A familiar story

- A team tunes 500 configurations of a gradient boosting model for customer churn
- Best 5-fold CV ROC AUC: **0.912**. It goes into the slide deck for management.
- Three months in production: **0.89**
- A second team reports that their model "beats ours by 0.004 AUC"

Three questions for today:

1. How do we search 500 configurations **efficiently**?
2. Why was 0.912 **too optimistic**, and what should we have reported?
3. Is a 0.004 difference **real**, and can we reproduce either number?

Motivation

- Need to select among models
- Need to select hyperparameters
- Need to select among preprocessing methods
- All of these choices are made **on data**, so they can overfit

Conditional Hyperparameters

- Kernels: `gamma` only matters for the RBF kernel, `degree` only for the polynomial kernel
- Neural nets: the size of layer 3 only matters if there are 3 layers
- Pipelines: the parameters of a step only matter if the step is chosen

Model selection as hyperparameter optimization

- One big search with many conditional hyperparameters
- Types: categorical, integer, continuous, conditional
- Different scales and priors (log-uniform for `C`, `learning_rate`)
- **CASH**: Combined Algorithm Selection and Hyperparameter optimization
- A global optimization problem on a complex, mixed space

Issues with Grid Search

- Need to define the grid by hand
- Cost is exponential in the number of hyperparameters
- Wastes evaluations on unimportant dimensions

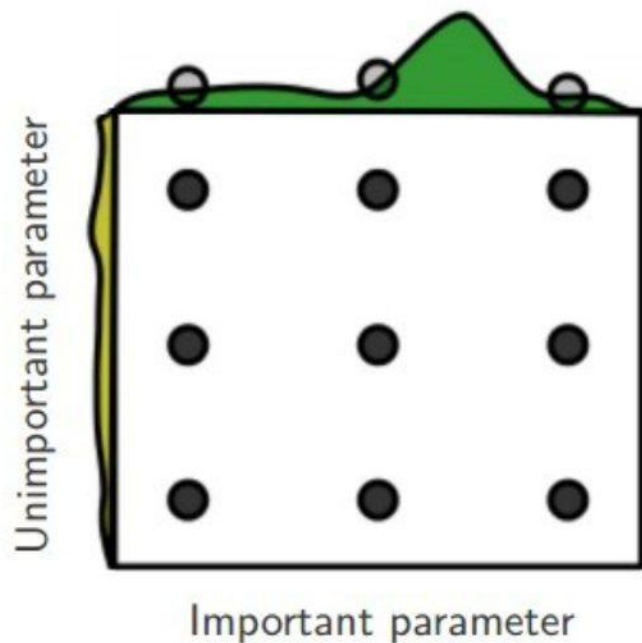
Black-Box Search Procedures

$$\Lambda^* = \arg \max_{\Lambda} f(\Lambda)$$

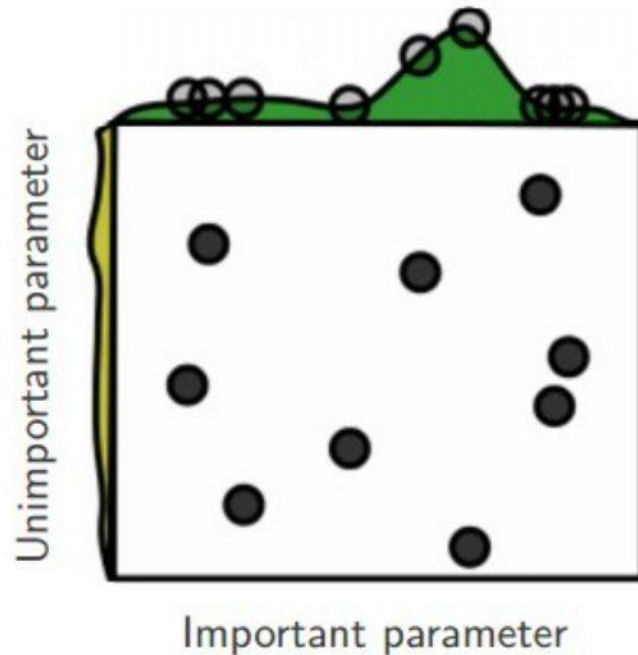
Parameters Λ , model evaluation f (e.g. mean CV score).

Random Search

Grid Layout



Random Layout



Random Search with scikit-learn

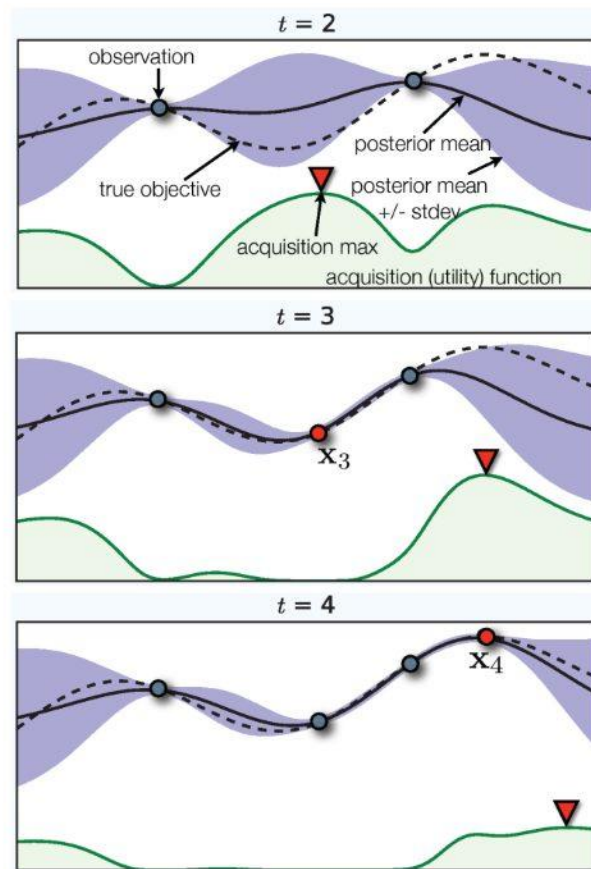
```
from scipy.stats import randint
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

param_dist = {"max_depth": [3, None],
              "max_features": randint(1, 11),
              "min_samples_split": randint(2, 11),
              "bootstrap": [True, False],
              "criterion": ["gini", "entropy"]}
random_search = RandomizedSearchCV(
    RandomForestClassifier(n_estimators=100), param_distributions=param_dist,
    n_iter=50, cv=5, random_state=0)
```

- lists or objects with an `rvs` method (`scipy.stats` distributions)
- Use continuous distributions (`loguniform`) for the biggest advantage

Bayesian Optimization, SMBO

- Fit a cheap probabilistic model (surrogate) to the evaluations so far
- Pick the next point by trading off exploration and exploitation
- The trade-off is encoded in an acquisition function (e.g. expected improvement)



Surrogate models

	Discrete Parameters	Scalable	Conditional Parameters	Papers & implementations	Specify parameters
Gaussian Process	?	X	?	many	bounds
Random Forest (SMAC)	✓	✓	✓	few	prior
Non- parametric (TPE)	✓	?	✓	few	prior

Bayesian optimization in practice: Optuna

```
import optuna
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_score

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 8, 128, log=True),
        "l2_regularization": trial.suggest_float("l2_regularization", 1e-6, 10, log=True),
    }
    model = HistGradientBoostingClassifier(**params, random_state=0)
    return cross_val_score(model, X, y, cv=5, scoring="roc_auc").mean()

study = optuna.create_study(direction="maximize",
                             sampler=optuna.samplers.TPESampler(seed=0))
study.optimize(objective, n_trials=50)
study.best_params, study.best_value
```

Implementations (2026)

- **Optuna**: TPE, GP, CMA-ES, multi-objective, pruning; our default in this course
- **SMAC3**: random forest and GP surrogates, Hyperband-style multi-fidelity
- **Ray Tune**: distributed execution, wraps Optuna and others
- **scikit-learn**: RandomizedSearchCV, HalvingGridSearchCV, HalvingRandomSearchCV
- Hyperopt (TPE) still works but sees little development; scikit-optimize, Spearmint and GPyOpt are archived

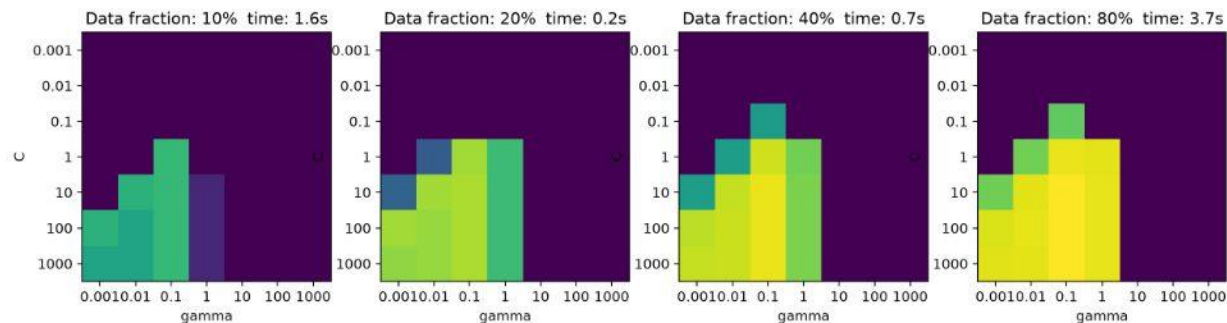
Beyond Black-Box

- Multi-fidelity optimization: use cheap approximations of f
- Meta-learning: use experience from other datasets
- Gradient-based hyperparameter optimization (mainly for neural nets)

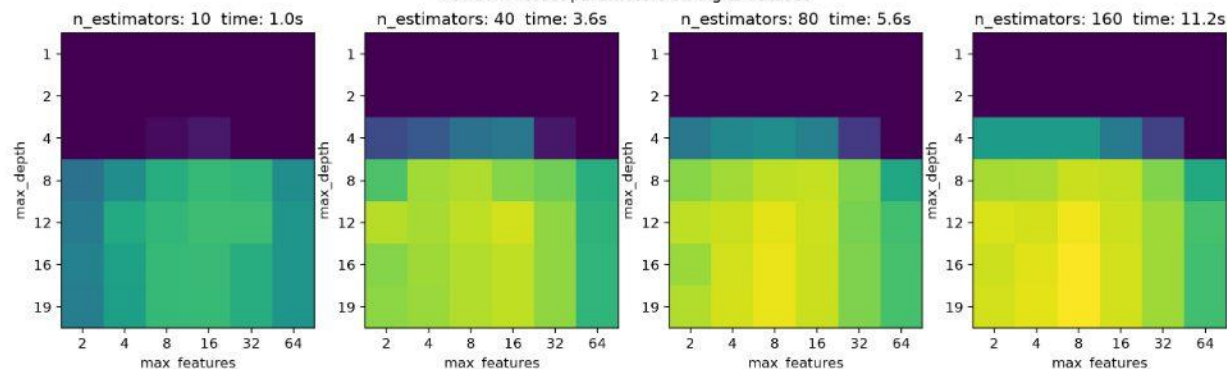
Multi-Fidelity Search

Approximate the function by a similar, cheaper function

RBF-SVM parameters on digits dataset



Random Forest parameters on digits dataset



Multi-Fidelity Optimization

- Budget can be: number of samples, number of trees or boosting rounds, epochs
- Evaluate many configurations cheaply, spend more only on promising ones
- Related to multi-armed bandits, with differences:
 - non-stationary: scores change as the budget grows
 - pulling an arm (evaluating) is expensive
 - possibly infinitely many arms (continuous parameters)

Successive Halving

- Start with n configurations and a small budget each
- Evaluate all, keep the best $1/\eta$ (e.g. $\eta = 3$: keep the best third)
- Multiply the budget of the survivors by η and repeat
- Stop when one configuration (or the full budget) is left

SUCCESSIVEHALVING (Finite horizon)

input: Budget B , and n arms where $\ell_{i,k}$ denotes the k th loss from the i th arm, maximum size R , $\eta \geq 2$ ($\eta = 3$ by default).

Initialize: $S_0 = [n]$, $s = \min\{t \in \mathbb{N} : nR(t+1)\eta^{-t} \leq B, t \leq \log_\eta(\min\{R, n\})\}$.

For $k = 0, 1, \dots, s$

 Set $n_k = \lfloor n\eta^{-k} \rfloor$, $r_k = \lfloor R\eta^{k-s} \rfloor$

 Pull each arm in S_k for r_k times.

 Keep the best $\lfloor n\eta^{-(k+1)} \rfloor$ arms in terms of the r_k th observed loss as S_{k+1} .

Output : $\hat{i}, \ell_{\hat{i},R}$ where $\hat{i} = \arg \min_{i \in S_{s+1}} \ell_{i,R}$

Successive Halving Example

- configurations $n = 81$, $\eta = 3$
- resource per configuration: number of training samples (max 10 000)

train 81 configurations with resources 41

train 27 configurations with resources 123

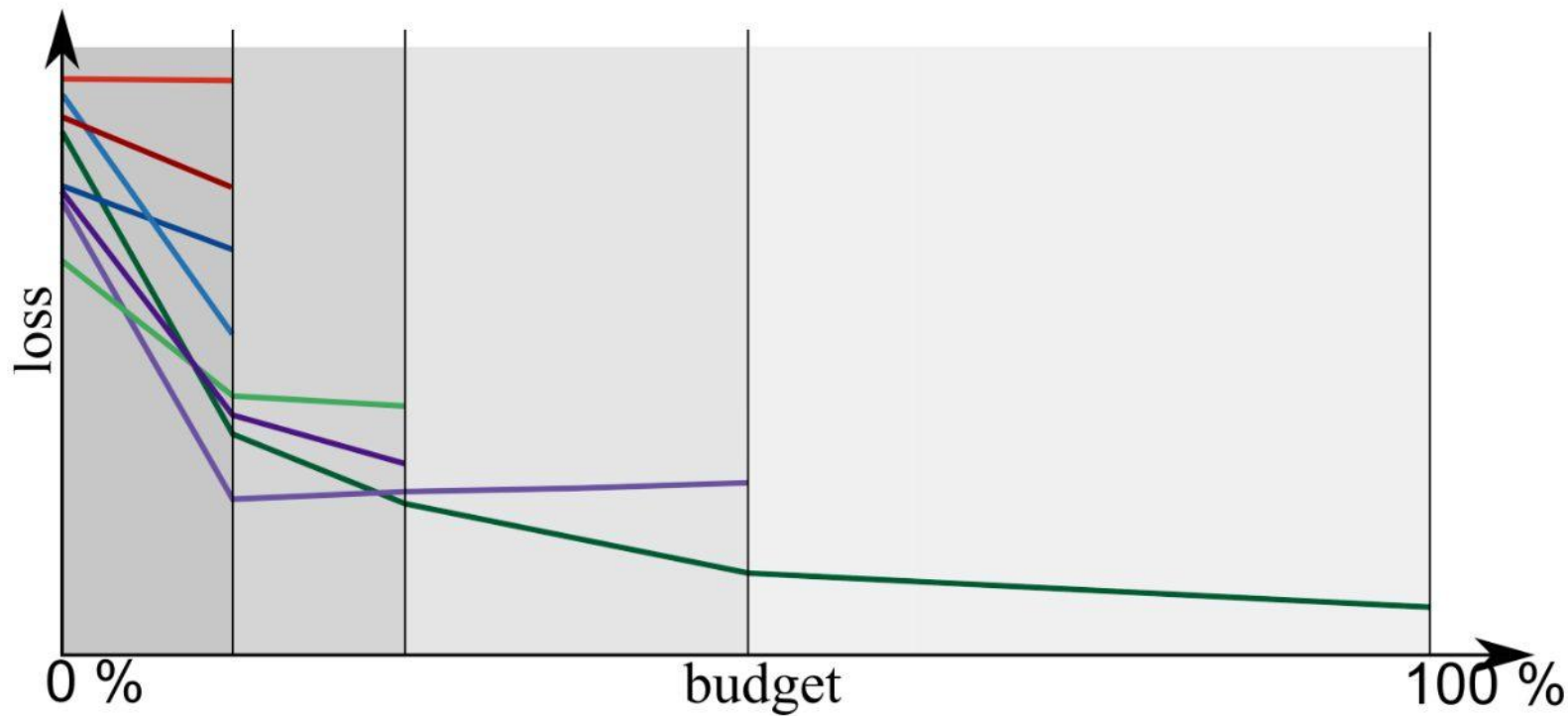
train 9 configurations with resources 370

train 3 configurations with resources 1111

train 1 configurations with resources 3333

resources total: 16638

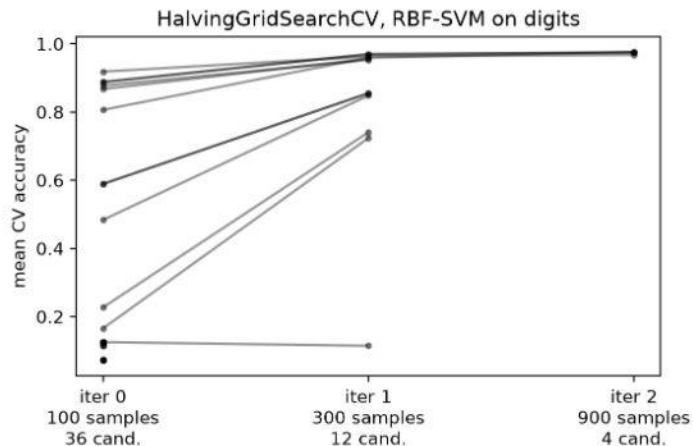
Successive Halving: learning curves



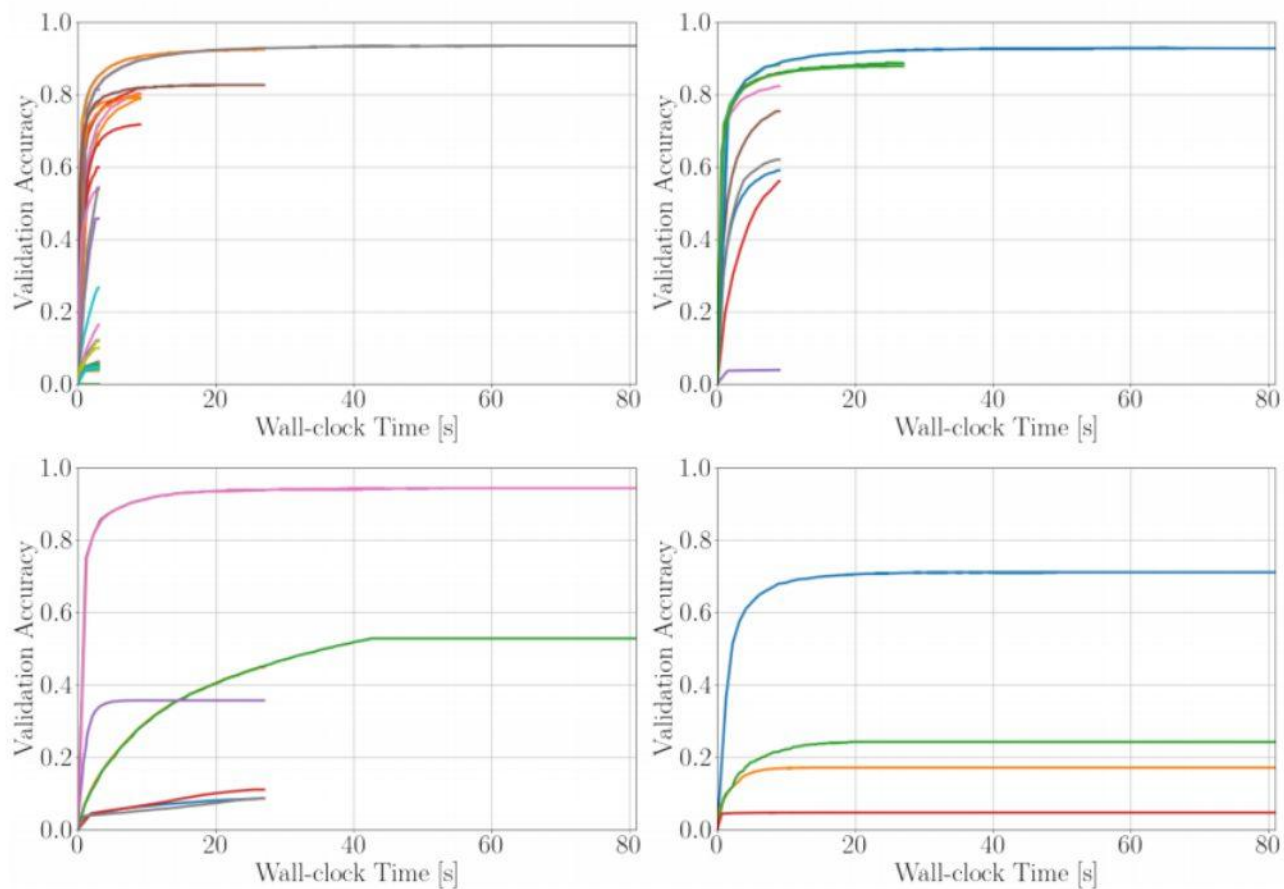
Successive Halving in scikit-learn

```
from sklearn.experimental import enable_halving_search_cv # noqa: F401
from sklearn.model_selection import HalvingGridSearchCV
from sklearn.svm import SVC

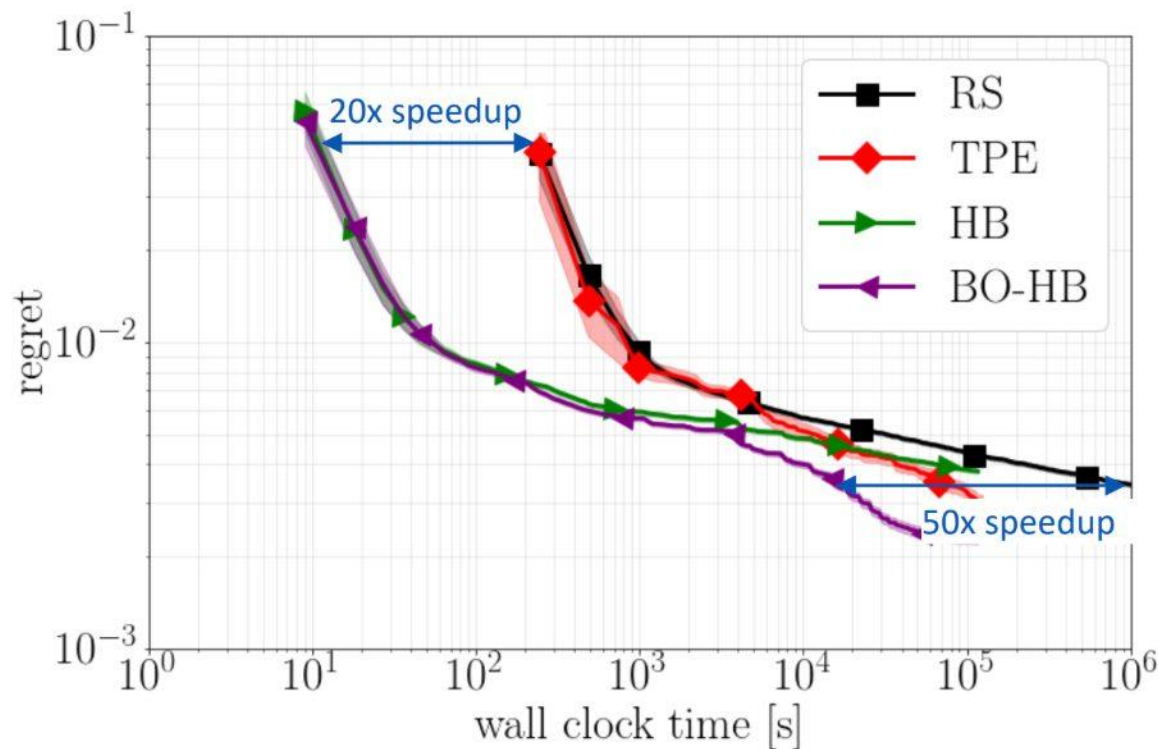
param_grid = {"gamma": np.logspace(-3, 2, 6), "C": np.logspace(-3, 2, 6)}
sh = HalvingGridSearchCV(SVC(), param_grid, factor=3,
                        resource="n_samples", random_state=0)
sh.fit(X_train, y_train)
sh.n_candidates_, sh.n_resources_ # [36, 12, 4], [100, 300, 900]
```



Hyperband



BOHB: Bayesian Optimization + Hyperband



Multi-fidelity in practice

"With the exception of the LeNet experiment (Section 3.3) and the 117 Datasets experiment (Section 4.2.1), the most aggressive bracket of SuccessiveHalving outperformed Hyperband in all of our experiments."

Li et al., JMLR 2018

- scikit-learn: `HalvingGridSearchCV`, `HalvingRandomSearchCV`
- Optuna: pruners (`SuccessiveHalvingPruner`, `HyperbandPruner`) stop bad trials early
- Needs a budget where low fidelity is informative

Meta-learning

Learning from experience (other datasets)

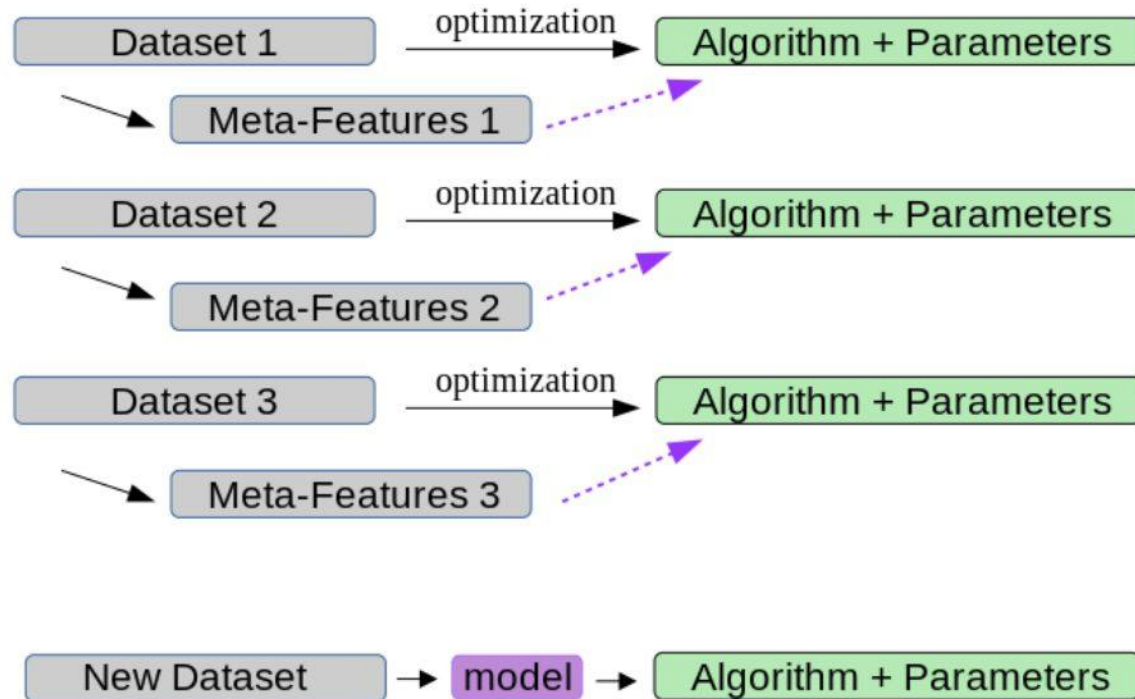
Ranking

- Run many algorithms on a large collection of datasets (e.g. OpenML)
- Rank by "best on average"
- Try the algorithms in that order

Portfolios

- Choose a small **diverse** set of configurations such that at least one is good on most datasets
- Optimal selection is a submodular optimization problem
- Greedy approximation works well
- Used in auto-sklearn 2.0 and AutoGluon

Meta-Features and Meta-Models



Active Testing and Recommendations

- Probabilistic matrix factorization for AutoML (Fusi et al., NeurIPS 2018)
- Datasets $\times$ pipelines performance matrix, treated like a recommender system

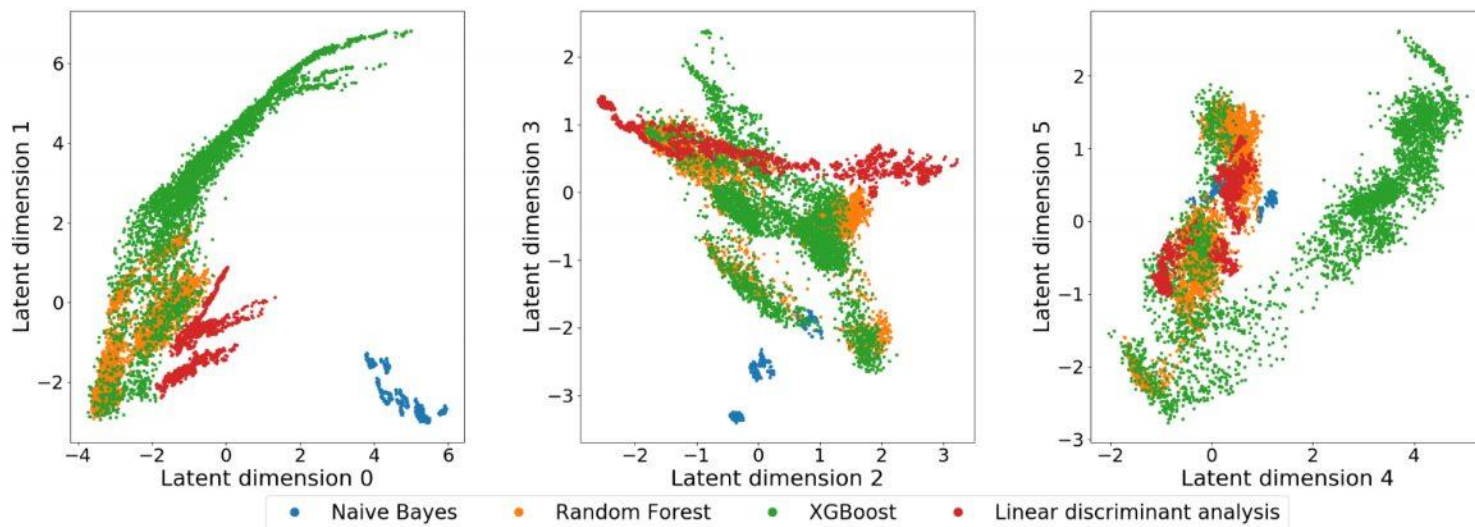
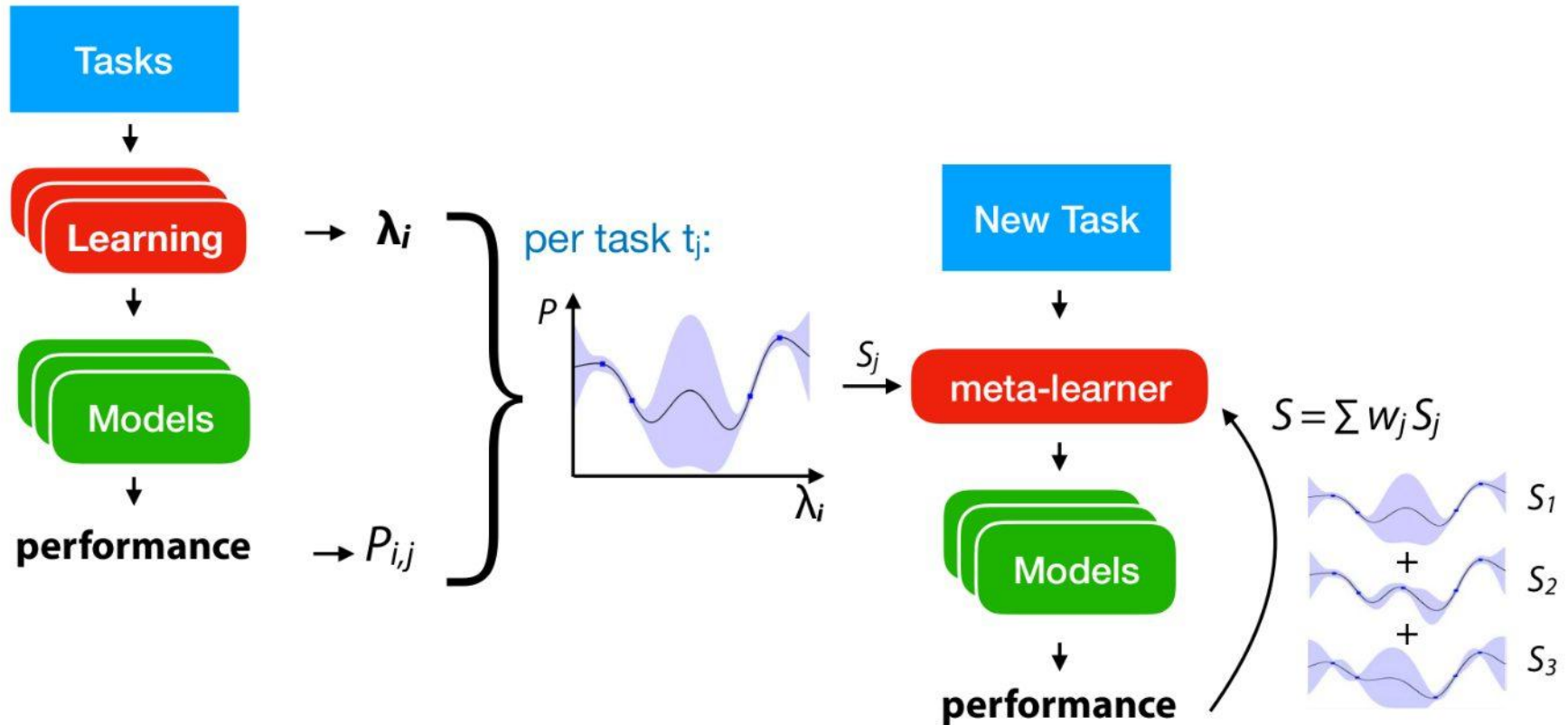


Figure 2: Latent embeddings of 42,000 machine learning pipelines colored according to which model was included in each pipeline. These are paired plots of the first 6 dimensions of our 20-dimensional latent space. The latent space effectively captures structure in the space of models.

Ensemble Models



AutoML systems today

- **AutoGluon** (Tabular): portfolio of strong models + multi-layer stacking; top of the AMLB benchmark
- **FLAML**: cost-frugal search, good for tight time budgets
- **auto-sklearn**: introduced meta-learning warm start + SMAC + ensemble selection; no longer actively maintained and does not install on current Python versions
- **TPOT**: genetic programming over pipelines
- Benchmark: AMLB, Gijsbers et al., JMLR 2024

A minimal AutoML: portfolio + successive halving

```
from scipy.stats import loguniform, randint
from sklearn.pipeline import Pipeline
from sklearn.model_selection import HalvingRandomSearchCV

pipe = Pipeline([("scale", StandardScaler()), ("clf", LogisticRegression())])
space = [
    {"clf": [LogisticRegression(max_iter=2000)], "clf__C": loguniform(1e-3, 1e2)},
    {"clf": [SVC()], "clf__C": loguniform(1e-2, 1e3),
     "clf__gamma": loguniform(1e-4, 1)},
    {"scale": ["passthrough"], "clf": [RandomForestClassifier(random_state=0)],
     "clf__max_features": randint(2, 30)},
    {"scale": ["passthrough"], "clf": [HistGradientBoostingClassifier(random_state=0)],
     "clf__learning_rate": loguniform(1e-2, 1)},
]
search = HalvingRandomSearchCV(pipe, space, n_candidates=60, factor=3,
                               random_state=0, n_jobs=-1).fit(X_train, y_train)
```

Practical recommendations

- Start with a strong default (gradient boosting) and a random or halving search
- Multi-fidelity (successive halving, pruning): simple and effective
- Optuna/TPE when evaluations are expensive or the space is conditional
- AutoML tools as a **baseline** to beat, not as a replacement for evaluation
- Log every trial (Part 2) and evaluate the whole procedure (next section)

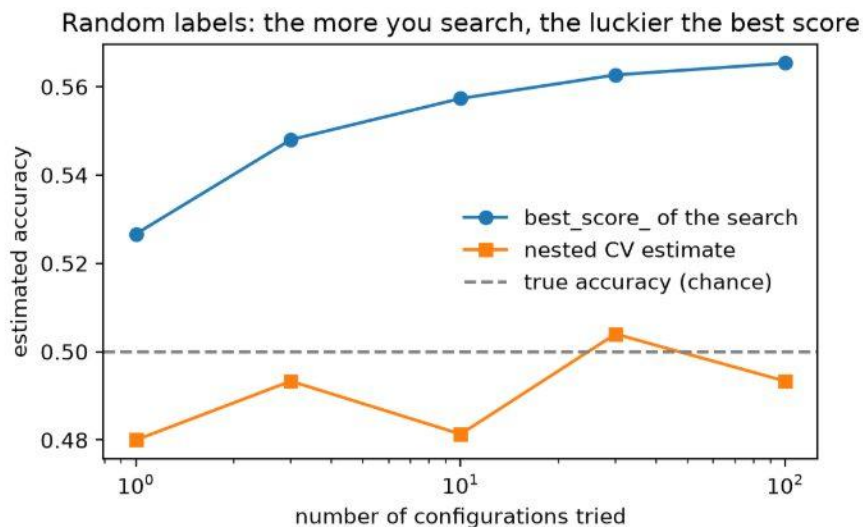
Criticisms of AutoML

- Do we need 100 classifiers?
- Do we need complex pipelines and ensembles?
- Creates large models that are hard to deploy and interpret
- "Making it too easy": skipping problem understanding and evaluation design

Evaluating a tuned model: nested cross-validation

The winner's curse

- `search.best_score_` is the **maximum** of many noisy CV estimates
- The maximum of noisy estimates is biased upward, even if all models are equally good
- The more configurations you try, the larger the bias
- Classic example: Kaggle public vs private leaderboard shake-ups



Nested cross-validation

- **Inner loop:** the search (e.g. `GridSearchCV`) selects hyperparameters
- **Outer loop:** evaluates the *whole search procedure* on data it never saw
``python from sklearn.model_selection import GridSearchCV,
StratifiedKFold, cross_val_score

```
inner = StratifiedKFold(5, shuffle=True, random_state=0) outer =  
StratifiedKFold(5, shuffle=True, random_state=1) search =  
GridSearchCV(make_pipeline(StandardScaler(), SVC()), param_grid,  
cv=inner)
```

```
nested_scores = cross_val_score(search, X, y, cv=outer) # 5 x (5 x 30) fits  
final_model = search.fit(X, y) # the model you deploy ``
```

Nested CV: worked example

Breast cancer data (569 samples), RBF-SVM, 6×5 grid over C , γ

Estimate	Accuracy
best_score_ of the grid search (non-nested)	0.981
Nested CV, 5 outer folds	0.975 ± 0.012

- Difference is small here: small grid, easy problem
- Grows with: search space size, small data, noisy metric
- Outer folds may pick **different** hyperparameters: that is expected

Statistical comparison of models

Is model A better than model B?

- CV gives J paired scores: a_j, b_j on the same splits
- Differences $d_j = a_j - b_j$; mean $\bar{d}$, variance σ_d^2
- Tempting: paired t-test, $t = \bar{d} / \sqrt{\sigma_d^2/J}$

Why this is optimistic

- Training sets overlap heavily (with 5-fold CV, any two share 75% of samples)
- Test sets in repeated CV overlap too
- So the d_j are **not independent**: σ_d^2/J underestimates the variance of $\bar{d}$
- Result: far too many "significant" differences (Dietterich, 1998; Nadeau & Bengio, 2003)

Corrected resampled t-test

Nadeau & Bengio (2003), *Machine Learning* 52:239–281

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) \sigma_d^2}}, \quad \text{df} = J - 1$$

- J : number of train/test splits (e.g. $r \times k$ for r -times repeated k -fold)
- $\frac{n_{\text{test}}}{n_{\text{train}}} = 1/(k-1)$ for k -fold CV
- The extra term accounts for the correlation between splits
- Recommended with repeated CV, e.g. 10×10 (Bouckaert & Frank, 2004)

Corrected t-test in code

```
import numpy as np
from scipy import stats

def corrected_ttest(a, b, n_train, n_test):
    """Nadeau-Bengio corrected resampled t-test for paired CV scores.
    d = np.asarray(a) - np.asarray(b)
    J = len(d)
    var = (1 / J + n_test / n_train) * d.var(ddof=1)
    t = d.mean() / np.sqrt(var)
    p = 2 * stats.t.sf(abs(t), df=J - 1)
    return t, p

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0)
s_lr = cross_val_score(lr_pipe, X, y, cv=cv, scoring="roc_auc")
s_rf = cross_val_score(rf, X, y, cv=cv, scoring="roc_auc") # same s
```

Worked example: logistic regression vs random forest

Breast cancer data, 10×5 -fold CV, ROC AUC

Mean AUC, logistic regression	0.9947
Mean AUC, random forest	0.9905
Mean difference $\bar{d}$	0.0042
Naive paired t-test	$t = 3.82, p = 0.0004$
Corrected resampled t-test	$t = 1.04, p = 0.30$

- Same data, same scores: the conclusion flips
- "No significant difference" $\neq$ "no difference": it means the data cannot tell

Many models, many datasets

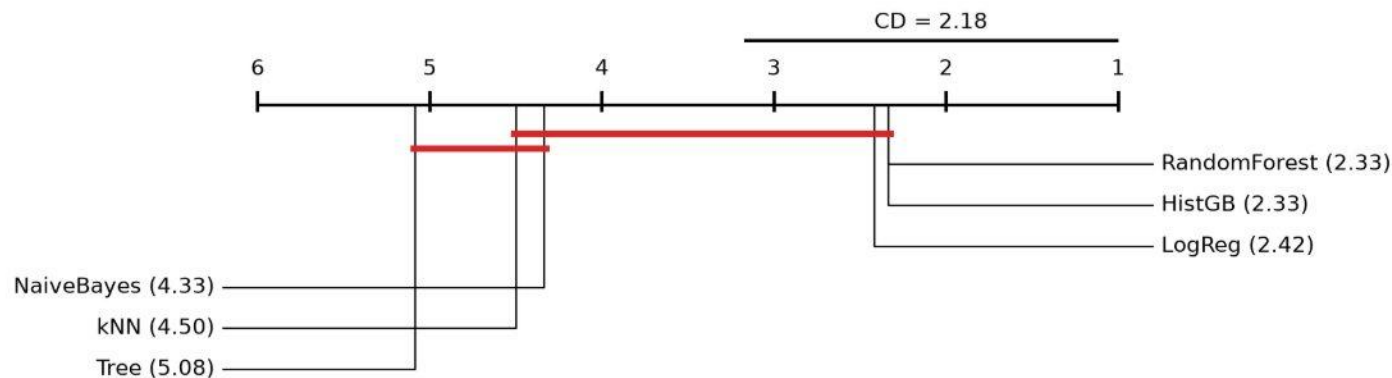
Demšar (2006), *JMLR* 7:1–30

- Rank the k models on each of N datasets (1 = best, ties averaged)
- **Friedman test**: are the average ranks R_j all equal?
- If rejected, **Nemenyi post-hoc test**: two models differ if their average ranks differ by more than

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}}$$

- q_α : studentized range statistic / $\sqrt{2}$ (2.850 for $k = 6$, $\alpha = 0.05$)
- Visualize as a **critical difference diagram**

Worked example: 6 models on 12 OpenML datasets



- Friedman: $\chi^2_F = 27.8$, $p < 10^{-4}$; CD = 2.18 with $N = 12$
- Random forest, gradient boosting and logistic regression are **not** distinguishable
- Only the clearest gaps (e.g. forests/boosting vs a single tree) are significant

Caveats and modern practice

- Nemenyi is conservative; pairwise Wilcoxon signed-rank tests with Holm correction are often preferred (Benavoli et al., JMLR 2016)
- Bayesian alternatives give probabilities of "A better", "equivalent", "B better" with a region of practical equivalence (Benavoli et al., JMLR 2017)
- Statistical significance is not practical significance: report effect sizes
- Fix the evaluation protocol **before** looking at results

Try it: predict the winner (15 min)

In `lab.ipynb`, section "Try it 1":

1. Before running anything, write down your predicted ranking of the six models on the 12 OpenML datasets
2. Add one model of your choice (e.g. a tuned SVC or an MLP from Week 6)
3. Run the benchmark, redraw the critical difference diagram
4. Which of your predictions were confirmed, and which differences are significant?

Part 2: Experiment Tracking with MLflow

Why track experiments?

- A tuning run produces hundreds of models: which one produced the number in your paper?
- Results depend on **code version + data version + parameters + random seed**
- Notebooks overwrite state; spreadsheets of results drift out of sync
- Tracking makes every result **traceable, comparable and reproducible**

Every number in your paper must trace back to a run ID.

MLflow Tracking: core concepts

- **Experiment:** a named group of runs (e.g. one research question)
- **Run:** one execution of your code
- Each run records:
 - **Parameters:** hyperparameters, preprocessing choices
 - **Metrics:** scores, possibly per step or per fold
 - **Artifacts:** plots, models, prediction files
 - **Tags:** Git commit, dataset hash, author
- UI: `mlflow ui` → compare runs side by side

Logging a run

```
import mlflow
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_score

mlflow.set_experiment("churn-baselines")

params = {"learning_rate": 0.1, "max_leaf_nodes": 31}
with mlflow.start_run(run_name="hgb-default"):
    mlflow.log_params(params)
    model = HistGradientBoostingClassifier(**params, random_state=0)
    scores = cross_val_score(model, X, y, cv=5, scoring="roc_auc")
    mlflow.log_metric("cv_roc_auc_mean", scores.mean())
    mlflow.log_metric("cv_roc_auc_std", scores.std())
    mlflow.set_tag("git_commit", git_commit)
```

Autologging and datasets

```
mlflow.sklearn.autolog()           # params, metrics, model logged aut  
  
with mlflow.start_run():  
    dataset = mlflow.data.from_pandas(df, name="churn-v2", targets="c  
    mlflow.log_input(dataset, context="training")  
    model.fit(X_train, y_train)
```

- `autolog` also records the fitted model and its signature
- `log_input` stores the dataset's name, schema and digest: **data version**

Tracking a hyperparameter search

```
import optuna

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0
        "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 8, 128,
    }
    with mlflow.start_run(nested=True):
        mlflow.log_params(params)
        score = cross_val_score(
            HistGradientBoostingClassifier(**params, random_state=0),
            X, y, cv=5, scoring="roc_auc").mean()
        mlflow.log_metric("cv_roc_auc", score)
    return score

with mlflow.start_run(run_name="optuna-hgb"):
    optuna.create_study(direction="maximize").optimize(objective, n_t
```

Comparing runs in pandas

```
runs = mlflow.search_runs(
    experiment_names=["churn-baselines"],
    filter_string="metrics.cv_roc_auc_mean > 0.85",
    order_by=["metrics.cv_roc_auc_mean DESC"])
runs[["run_id", "tags.mlflow.runName", "params.learning_rate",
      "metrics.cv_roc_auc_mean"]].head()

client = mlflow.MlflowClient()
fold_scores = [m.value for m in client.get_metric_history(run_id, "fo
```

- `search_runs` returns a DataFrame: columns `params.*`, `metrics.*`, `tags.*`
- Per-fold scores logged with `step=` come back with `get_metric_history`
- Feed two runs' fold scores into the corrected t-test

Try it: a tuning competition (15 min)

In `lab.ipynb`, section "Try it 2":

- Churn data, fixed train/test split; the test set is locked
- Budget: at most **20 CV evaluations** (Optuna, random or halving search)
- Log every trial to MLflow under your name as a tag
- Report your best CV score, then unlock the test score
- Class discussion: whose CV score was most optimistic, and did the winner by CV win on test?

Try it: is it significant? (10 min)

In `lab.ipynb`, section "Try it 3":

1. Use `mlflow.search_runs` to find the two best runs of the experiment
2. Retrieve their per-fold scores with `get_metric_history`
3. Compare them with the naive paired t-test and the corrected resampled t-test
4. Write one sentence you would put in a paper about this comparison

Good tracking practice

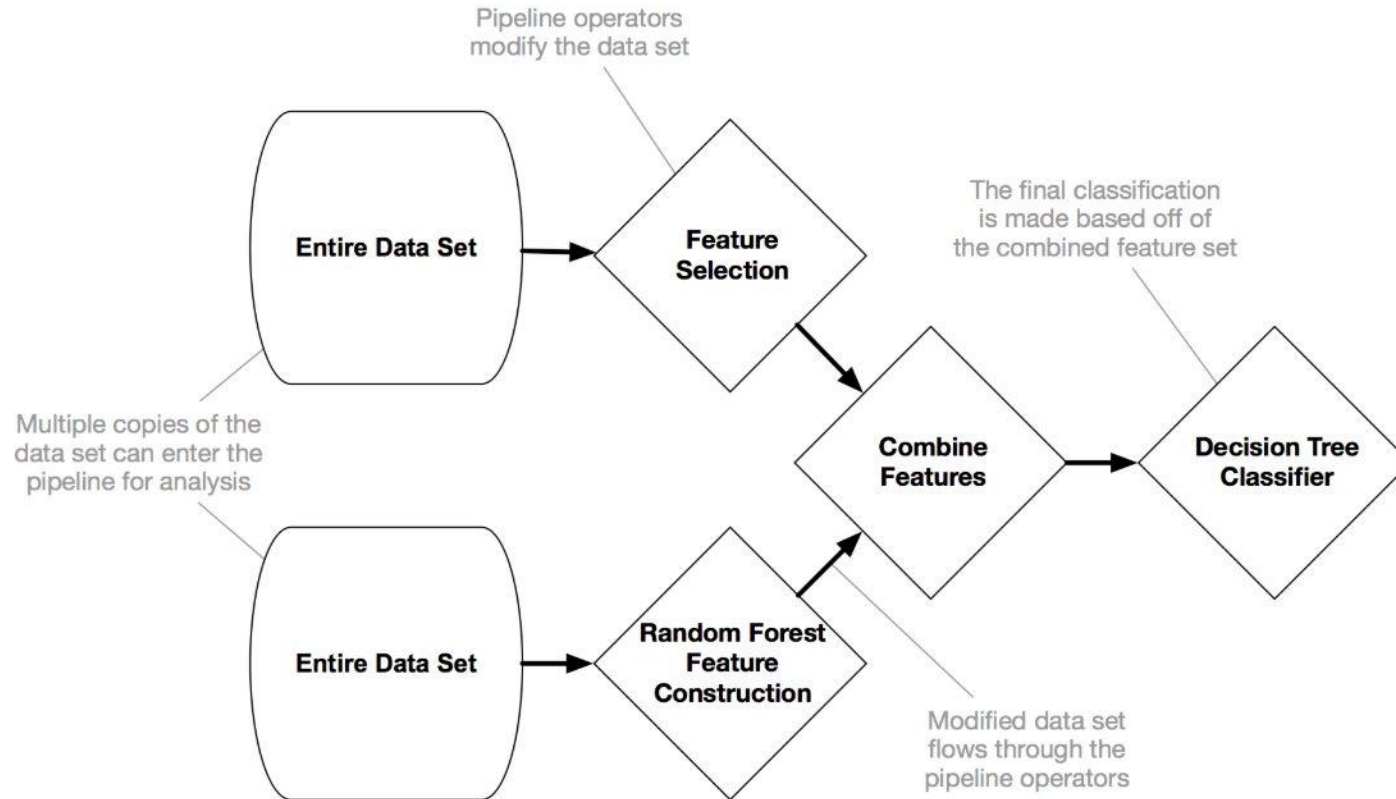
- One experiment per research question; descriptive run names
- Always log: **seed**, **Git commit**, **dataset digest**, **CV scheme**
- Log per-fold scores, not only the mean: you need them for statistical comparison
- Save the config, not just the result
- Never report a number you cannot find in the tracking server

Course rule: from Week 2 on, every notebook and homework logs to MLflow.

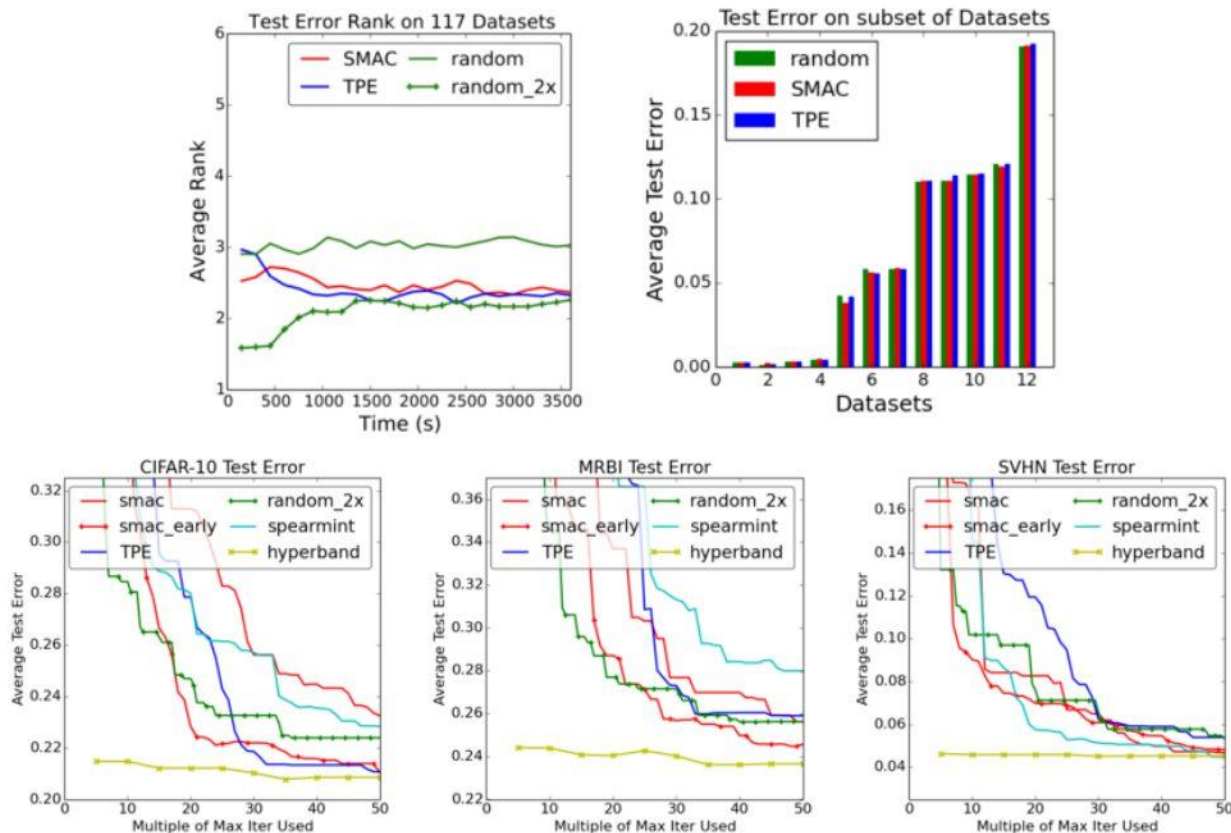
Questions?

Backup

Evolutionary Methods: TPOT



How much better than random?



Recht & Jamieson, argmin.net blog, 2016; data from Li et al., JMLR 2018

Hyperband

Algorithm 1: HYPERBAND algorithm for hyperparameter optimization.

```
input      :  $R, \eta$  (default  $\eta = 3$ )
initialization:  $s_{\max} = \lfloor \log_{\eta}(R) \rfloor, B = (s_{\max} + 1)R$ 
1 for  $s \in \{s_{\max}, s_{\max} - 1, \dots, 0\}$  do
2    $n = \lceil \frac{B}{R} \frac{\eta^s}{(s+1)} \rceil, \quad r = R\eta^{-s}$ 
   // begin SUCCESSIVEHALVING with  $(n, r)$  inner loop
3    $T = \text{get\_hyperparameter\_configuration}(n)$ 
4   for  $i \in \{0, \dots, s\}$  do
5      $n_i = \lfloor n\eta^{-i} \rfloor$ 
6      $r_i = r\eta^i$ 
7      $L = \{\text{run\_then\_return\_val\_loss}(t, r_i) : t \in T\}$ 
8      $T = \text{top\_k}(T, L, \lfloor n_i/\eta \rfloor)$ 
9   end
10 end
11 return Configuration with the smallest intermediate loss seen so far.
```

- Outer loop: brackets s from aggressive (many configs, small budget) to conservative
- Inner loop: successive halving within each bracket

Discretization

Are continuous parameters actually important?

- Anecdotal evidence and portfolio results suggest a few well-chosen values are often enough

Multi-Task Bayesian Optimization

- Build one surrogate over all datasets at the same time
- Hard to scale with Gaussian processes
- Neural and tree-based surrogates scale better

Although we already reduced the space of considered ML algorithms substantially compared to our previous Auto-sklearn (4 vs. 15 classifiers), we could have reduced this set even further since, in the end, only XGBoost models ended up in the final ensembles for the challenge.

Feurer et al., PoSH auto-sklearn